```
$carData = ['make' => 'Audi', 'model' => 'Golf', 'variant' => 'VRs'];
$correctData = ['make' => 'VW', 'variant' => 'GTI'];

$carData = array_replace($carData, $correctData);

print_r($carData);

Output:
Array
(
    [make] => VW
    [model] => Golf
    [variant] => GTI
)
```

`array_replace()` takes two arrays as arguments and builds a new one from them. The input array keys are compared, and if the keys match, the element from the second array is used; otherwise, it will be the element from the first. As we can see from the example, `$correctData` contains two keys that are also present in `$carData`, so the values for `make` and `variant` are `VW` and `GTI`, respectively. As the `model` isn't present in `$correctData`, the value from `$carData` is used instead.

This function can take multiple replacement arrays (e.g., `array_replace($array, $replace1, $replace2, $replace3);`). It will perform the same key checking, the rule being that if multiple instances of a key are found in the replacement arrays, it is always the last one encountered that is used in output. If `$replace2` and `$replace3` both have a key called `foo`, it is `$replace3['foo']` that will be used in output.

Comparing Arrays

We have seen multiple examples so far of accessing array elements for use in various statements. All the usual rules of variable comparison apply to those elements, be they numbers, strings, booleans, etc. It is also possible to compare entire arrays to each other in a single statement.